

Dynamic Successor Features for transfer learning and guided exploration

Norman Tasfi^a, Eder Santana^b, Luisa Liboni^{a,*}, Miriam Capretz^a

^a Department of Electrical and Computer Engineering, Western University, Canada

^b Twitch Tv Inc., United States of America

ARTICLE INFO

Article history:

Received 11 October 2022

Received in revised form 13 February 2023

Accepted 14 February 2023

Available online 2 March 2023

Keywords:

Reinforcement learning

Successor features

Transfer learning

Deep learning

ABSTRACT

The Successor Feature framework for Reinforcement Learning algorithms improves task transfer by decomposing the learned state–action value function. The decomposition involves two components, one that captures future-expected state features and the other that models the task-related reward structure. However, successful transfer between tasks depends heavily on how the reward function changes, possibly leading to failure of the original Successor Feature formulation. This paper proposes the Dynamic Successor Feature framework, DynSF, by extending the mathematical formulation of the original Successor Feature framework to center around a learned state-transition model. Under this formulation, the state-transition model dynamically induces the acting policy. The flexibility of DynSF also extends to the architecture, requiring only a state-transition model and a small vector of parameters. This architecture provides immense flexibility in the choice of the model used to learn the state-transition model. The DynSF framework is evaluated and compared to other baseline algorithms through several experiments in a continuous grid world environment, a robotic Reacher, and pixels in the Doom environment.

© 2023 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

The performance of Reinforcement Learning (RL), which consists of learning through goal-oriented interactions of an agent with the environment, has improved by leaps and bounds over the past few years, outperforming human experts on gaming tasks such as Go [1], Poker [2], and Atari [3,4]. RL has also been successfully used to tackle problems in robotics [5,6], fluid mechanics [7,8], energy [9,10], chip placement [11], and other areas, extending to applications outside the computer science and engineering fields.

A well-known challenge in the field of RL is sampling inefficiency, as described by Sutton and Barto [12] and Zhang et al. [13], where the agent must learn from scratch through an immense amount of data or save manual labeling efforts by training on labeled data available in a related source domain. Transfer learning in RL aims to leverage the knowledge of previous models to ease learning through less data, avoiding concept drift [14]. In RL, the decisions are grounded through a reward function. More specifically, the central question in RL transfer learning is the transfer of knowledge between tasks in an environment when only the reward specification changes, but other environment

characteristics remain fixed. Previous works present reusing RL models to initialize learning across different reward specifications, leading to improved learning speed. Some examples of RL transfer learning are in the fields of gaming and self-driving cars [15,16].

The Successor Feature (SF) framework presented by [17] computes a representation of the environment that can be transferred across different reward functions. In the SF framework, the state–action value function is expressed as the dot product between a vector of expected discounted future-state occupancies (the Successor Features) and another vector representing the immediate reward in each of those successor states [18–28]. During the transfer, the SFs are held frozen, and only the parameters from the reward component are trained, which requires fewer samples.

The SF framework was developed to tackle task transfer but also brings light to the mathematical formulation of RL by partially disentangling the environment from task-related rewards. By taking advantage of the SF model, this paper explores the SF framework's mathematical formulation to facilitate understanding of what is learned. The SF framework formulation has been extended to yield the Dynamic Successor Feature (DynSF). The process of doing this extension sheds light on what the original SF framework learns, showing mathematically that the original SF representation is tied to marginally learning a specific policy, which governs the SFs under a fixed discount factor.

* Corresponding author.

E-mail addresses: ntasfi@uwo.ca (N. Tasfi), eder.santana@twitch.tv (E. Santana), luisa.liboni@uwo.ca (L. Liboni), mcapretz@uwo.ca (M. Capretz).

List of Symbols

a_t	Action at time t
$\bar{\pi}$	Arbitrary policy
γ	Discount factor
G_t	Discounted sum of rewards at time t
ϕ_t	Encoded state features at time t
$\mathcal{L}_{ED}$	Encoder–decoder loss
π	Ongoing policy
s'	Proceeding state
r_t	Reward at time t
$\mathcal{L}_r$	Reward loss
w	Reward weight vector
Φ	Set of all encoded valid states in S
$\mathcal{A}$	Set of all possible actions
S'	Set of all possible proceeding states
S	Set of all possible states
s_t	State at time t
$\mathcal{M}$	State transition model
Q^π	State–action value function
$\mathcal{L}_{\mathcal{M}}$	State-transition model loss
ψ	Successor features

In fact, after training, the SFs are tied to the optimal policy learned. However, when transferring to another task, the ongoing optimal policy might not be the same [29]. For the SF framework, a successful transfer depends on how drastically the reward function changes between tasks. If a slight change in rewards occurs, then the optimal policy change is not drastic, and relearning can converge within a good response time. Otherwise, if a significant change occurs, then the original SF framework can indeed fail to transfer.

In particular, it will be shown that through DynSF, SFs can be modeled using a state-transition model. In this newly proposed model, by using rollouts of state embeddings that are learned and output by the state-transition model to induce a policy, it is possible to adjust an agent dynamically. This induced policy and the discount factor can be parameterized on the fly during a rollout. Dynamic adjustment of the discount factors and acting policy is possible because they are treated as functional parameters of the state-transition model. This novel formulation makes it possible to use different policies and discount factors during training and testing. In this sense, the DynSF framework can be used not only for transfer learning but also for guided exploration [30], where prior knowledge is used regarding the given environment and essentially implies that some one-step information can be used to rule out suboptimal actions, including those actions constrained to safety on a safe RL setup [31].

The proposed approach can be related to model-based RL, where an environment model is learned to improve action efficiency. In such model-based approaches, action efficiency is improved by using the environment model to simulate future actions and optimize the chosen action [32–34]. The DynSF framework proposed in this study is most similar to this action planning approach, but with the induced policy and discount factor being dynamically created, which brings flexibility and efficiency to training and transfer.

The DynSF framework combines supervised learning of a state-transition model and the reward component, enabling several supervised machine learning techniques and being able to tackle able to exploit the challenge of task relatedness in RL [35]. As a result, the DynSF formulation improves the fidelity of the captured

state dynamics over those of SFs through a more expressive state-transition model. Now, novel algorithms for supervised learning can be used to learn the rewards correctly.

The contribution of this paper can be summarized as follows:

- DynSF enables a state-transition model to be used for state rollouts where the discount factor and policy can be set on the fly, which is advantageous compared to common model-based approaches.
- The paper examines how rollout length and policy affect the performance of DynSF to compare its performance to the original framework and baselines.
- Through experiments, the flexibility of DynSF is analyzed, where policy and discount factors can be parameterized dynamically and where different supervised ML algorithms can be used to learn the state transition model.
- A significant task change is used to show that DynSF performs better during task transfer than SF. DynSF's learned state model is shown to offer a lesser bias than the previously learned policy.

After the introduction of relevant background material in Section 2, DynSF is introduced in Section 3. A brief review of the environments and experiments used to validate the DynSF model is presented in Section 4. Section 5 contains experimental results supporting the flexible representation of the DynSF model and a discussion of these results. The paper concludes in Section 6 with a final discussion on the contributions of this research and possible avenues for future work. Finally, the Methods chapter, found after the references, discloses in detail the methods used in the interest of research reproducibility.

2. Background

This section formally defines the RL problem and addresses the underlying mathematical formulation necessary to derive the SF framework, which is extended to yield the DynSF framework.

2.1. Reinforcement learning

Assume that an agent interacts with an environment according to a Markov Decision Process (MDP) [36]. At each time step t , the agent observes a state $s_t \in S$, where S is the set of all available states. From state s_t , the agent selects an action a_t from the set of all available actions $\mathcal{A}$ and executes it in the environment.

The environment transitions to a new state s_{t+1} and emits a reward r_t according to the environment transition dynamics $P(s_{t+1}, r_t | s_t, a_t)$. This paper focuses on tasks and environments where only the rewards are statistically independent of the previous state, being entirely defined by the resulting state only. Ma et al. [20] present several tasks where this assumption holds.

The objective of the agent is to find a policy $\pi(s) : S \mapsto \mathcal{A}$ that maximizes the expected value of the discounted sum of rewards G_t in the environment, defined as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k},$$

where $\gamma \in [0, 1]$ is the discount factor that trades off the importance of immediate and future rewards. A policy π can be derived from the state–action value function, the Q-Learning function, which is defined as:

$$Q^\pi(s, a) = \mathbb{E}^\pi[G_t | s_t = s, a_t = a], \quad (1)$$

which provides the future expected reward of selecting any action $a \in \mathcal{A}$ and then following the policy $\pi(s)$ afterwards, such that $\pi(s) \in \arg\max_a Q^\pi(s, a)$.

2.2. Successor feature framework

The SF framework decomposes the Q-Learning function into two components. The decomposition is accomplished by first assuming that the reward r_t is computed as the dot product between state features $\phi_t \in \mathbb{R}^z$, extracted from s_t using an encoder $E(s) : \mathcal{S} \rightarrow \Phi$, and a vector $w \in \mathbb{R}^z$, that models the reward of the task, where z is a hyperparameter that controls the embedding dimension of the vectors.

Therefore, from the assumption that $r_t = \phi_t^\top \cdot w$ and the definition of the Q-Learning function in (1), the following formulation is obtained:

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}^\pi [r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t = s, a_t = a] \\ &= \mathbb{E}^\pi [\phi_t^\top \cdot w + \gamma \phi_{t+1}^\top \cdot w + \dots | s_t = s, a_t = a] \\ &= \psi^\pi(s, a)^\top \cdot w, \end{aligned} \quad (2)$$

where vector $\psi^\pi(s, a) \in \mathbb{R}^z$ is referred to as the SFs under policy $\pi(s)$, with one vector defined *per* action. The i -th component of $\psi^{\pi(i)}(s, a)$ predicts the future expected discounted sum of the feature $\phi_t^{(i)}$, which is a function of the state s_t that the policy $\pi(s)$ visits. As mentioned above, this paper focuses on tasks where the reward r_t is a function of only the current state s_t and not the proceeding state s' : $\mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}'$.

For brevity, $\psi^\pi(s, a)$ is referred to simply as ψ_t^π and policy $\pi(s)$ as π . Under this decomposition, transfer between new tasks requires retraining of just the smaller reward vector w while keeping the state features ϕ_t and the SFs ψ_t^π frozen, reusing previously learned knowledge.

3. DynSF – Dynamic Successor Feature Framework

This section presents the proposed variant of the SF framework, which uses a state-transition model to bootstrap the SFs dynamically. The definition of the SFs ψ_t^π is as follows:

$$\psi_t^\pi = \mathbb{E}^\pi [\phi_t + \gamma \phi_{t+1} + \gamma^2 \phi_{t+2} + \dots], \quad (3)$$

where the visited latent states within the expectation depend on the particular policy π . From this, (3) can be rewritten in terms of a chain of state transitions dictated by the stated policy. Defining (3) in terms of state transitions requires access to a model of the environment that can predict the next latent state ϕ_{t+1} given the current state ϕ_t and action a .

This study refers to $\mathcal{M}$ as the latent-state transition model and defines it as:

$$\mathcal{M} : (\phi_t \in \Phi) \times (a_t \in \mathcal{A}) \mapsto (\phi_{t+1} \in \Phi),$$

where Φ is the set of all encoded valid states in $\mathcal{S}$. Following this definition, (3) can now be rewritten with the latent state-transition model $\mathcal{M}$ as follows:

$$\psi(s, a, \bar{\pi}, \gamma) = \phi_t + \gamma \mathcal{M}(\phi_t, \bar{\pi}(\phi_t)) + \dots, \quad (4)$$

where ψ_t^π now loses the superscript π because it accepts additional arguments such as any policy $\bar{\pi}(\phi_t)$, working on the latent space representation, and the discount factor γ . Rewriting (4) with a summation and a k -length rollout yields:

$$\psi(s, a, \bar{\pi}, \gamma) = \phi_t + \gamma \mathcal{M}(\phi_t, \bar{\pi}(\phi_t)) + \dots + \sum_{j=1}^{k-1} \gamma^{j+1} \mathcal{M}(\hat{\phi}_{t+j}, \bar{\pi}(\hat{\phi}_{t+j})), \quad (5)$$

where $\hat{\phi}_{t+1} = \mathcal{M}(\phi_t, \bar{\pi}(\phi_t))$ and $\hat{\phi}_{t+j+1} = \mathcal{M}(\hat{\phi}_{t+j}, \bar{\pi}(\hat{\phi}_{t+j}))$. Eq. (5) implies that the SFs can be computed dynamically given a latent state-transition model, a discount factor γ , and a policy $\bar{\pi}$. The SF function $\psi(s, a, \bar{\pi}, \gamma)$ is bootstrapped before each step in

the environment by gradually unrolling the latent space with $\mathcal{M}$, as shown in (5). Comparing (3) and (5) reveals that the original function formulation of ψ_t^π in (3) is learning an amortized version of this rollout under a specific policy $\pi = \bar{\pi}$ and discount factor γ .

The final Q-Learning function in (2) can now be dynamically inferred based on $\bar{\pi}$ and the discount factor γ at each roll-out by $\mathcal{M}$ for k steps:

$$Q(s, a, \bar{\pi}, \gamma) = \{\phi_t + \gamma \mathcal{M}(\phi_t, \bar{\pi}(\phi_t)) + \dots + \sum_{j=1}^{k-1} \gamma^{j+1} \mathcal{M}(\hat{\phi}_{t+j}, \bar{\pi}(\hat{\phi}_{t+j}))\}^\top \cdot w. \quad (6)$$

This contrasts with the traditional phrasing of the SF framework, and some important considerations of this new formulation are made next.

Rollout length. Given the DynSF formulation, note that the original SF function learns to combine information about the policy, discount factor and state transitions in some marginalized form. It is possible to claim a mathematical equivalence between the original SF framework and DynSF, given that an equivalent fixed policy π is used throughout the rollouts in (6), and that the number of rollouts matches the task state horizon. Although DynSF brings flexibility, the chosen rollout length certainly affects performance, as shown and discussed in Section 5.

Rollout policy. When the SF function $\psi(s, a, \bar{\pi}, \gamma)$ is being dynamically computed, as described in (5), an acting policy $\bar{\pi}$ is required. The policy can be passed in as a function argument, providing immense flexibility. An experiment involving the deployment of different acting policies is discussed in Section 5. This can have significant implications, for example, for guided and safe exploration/ At its core is an optimization formulation in which the constraints restrict attention to a subset of safe policies and state visitations with some well-controlled probability [37].

3.1. Model learning

The proposed model is composed of the following parts: an encoder $E(s) : \mathcal{S} \rightarrow \Phi$, a decoder $D(\phi) : \Phi \rightarrow \mathcal{S}$, a state-transition model $\mathcal{M}(\phi, a)$, and a reward vector w .

The state-transition model is trained to predict the next latent state ϕ_{t+1} given the current state ϕ_t and action a_t and minimizes:

$$\mathcal{L}_{\mathcal{M}} = \mathbb{E}[(\phi_{t+1} - \mathcal{M}(\phi_t, a_t))^2],$$

where the target variable ϕ_{t+1} is produced by encoding the following state. State grounding, as used by [32], is used to ensure that the latent representation ϕ_t contains meaning about the environment and fits naturally into the framework. The reward prediction step keeps the same form used in the SF framework, which minimizes

$$\mathcal{L}_r = \mathbb{E}[(r_t - \phi_t^\top w)^2].$$

There is also a reconstruction loss where the model must learn to reconstruct the state s_t from a latent representation ϕ_t using the decoder $D(\phi_t)$:

$$\mathcal{L}_{ED} = \mathbb{E}[(s_t - D(\phi_t))^2].$$

By summing the losses together, the following sum of mean-squared error losses is minimized:

$$\mathcal{L} = \mathcal{L}_{\mathcal{M}} + \mathcal{L}_r + \mathcal{L}_{ED}.$$

Unlike Kulkarni et al. [19], all model components together are minimized at once and end-to-end instead of requiring interleaved updates.

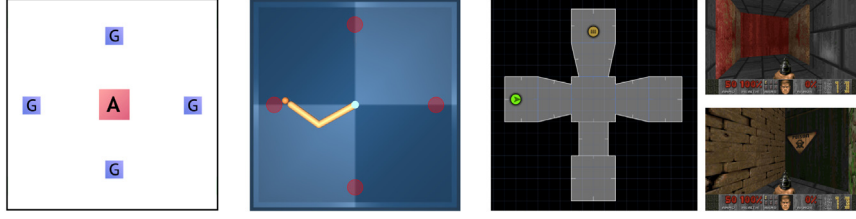


Fig. 1. Visualizations of the environments used in this study. From left to right: Axes environment, Reacher, Doom. **Axes (left):** The agent shown in red must traverse to the various goals shown as colored “G”s. **Reacher (center):** The agent controls the robotic arm shown and must move the arm’s end to a goal position shown by the colored circles. **Doom (right):** The agent must traverse between rooms looking for a goal point. The map layout is shown on the right, and images of the environment are on the far right.

Because the training formulation is quite general, any state-transition model can be dropped into place with its respective losses. This implies the flexibility to use any supervised learning model, which is considered, together with the acting policy and discount factor flexibility on the fly, to be one of the advantages of this new formulation compared to other model-based learning approaches.

4. Environments and experiments

This section provides details on the environments against which the proposed methodology was tested, the reasoning behind the choice of the environments, and a description of the experimental methodology.

The environments were chosen to guarantee comparability and research reproducibility within previous studies on SFs [17]. The following subsection describes Axes, a navigation task, Reacher, a robotic control task built using the MuJoCo engine [38], and the 3D maze *Doom* game engine. Fig. 1 shows these environments, which act as test beds for the DynSF method. Each contains tasks specified by goal location and split between training and test distributions.

4.1. Environments

4.1.1. Axes

In this environment, shown in Fig. 1, the agent, spawned at a uniform location, must navigate the map to reach a goal location in as few steps as possible by moving in each of the cardinal directions: *up*, *down*, *left*, and *right*. Four separate goal locations exist. An episode ends when either the agent reaches the goal or moves more than 25 steps in the environment. The agent’s initial position is randomly placed in a grid of 3×3 step units, centered at (0, 0).

In the commonly used version of this environment also used within other SF studies [17], the reward the agent sees is the negative squared distance between its location and the target goal. The reward function takes the form

$$\begin{aligned} & -d(p_a, p_g) \\ & = -\sqrt{(p_a^{(1)} - p_g^{(1)})^2 + (p_a^{(2)} - p_g^{(2)})^2 + \dots + (p_a^{(n)} - p_g^{(n)})^2}, \end{aligned} \quad (7)$$

where $d(p_a, p_g)$ is the distance between the agent position $p_a \in \mathbb{R}^n$ and the goal position $p_g \in \mathbb{R}^n$ within the environment. For Axes, $n = 2$, meaning that positions are in 2D.

Note that the agent receives a reward equal to the negative distance between itself and the goals at each step. The state space of the environment is $s_t \in \mathbb{R}^4$, and reward prediction can be accomplished by using a 1-hot encoded reward vector.

Given the 2D characteristic, this environment provides visual access to the state value over the entire state space to understand how the agent perceives the environment.

4.1.2. Reacher

This environment consists of a robotic control task defined in the MuJoCo physics engine [38], as shown in Fig. 1 (center). The agent must move a robotic arm to a specific point in space by activating four torque-controlled motors. Because this is a continuous control task, the actions were discretized so that the agent had nine discrete actions to control the arm movements. Therefore, the four-dimensional continuous action space $\mathcal{A}$ was discretized using two values per dimension: the maximum positive and maximum negative torque for each actuator. An all-zero option that applies zero torque along all actuators was included, resulting in a total of nine discrete actions.

As in the Axes environment, the reward function takes the form of (7), where for Reacher, $n = 3$, meaning that positions are 3D. Therefore, the agent receives a reward equal to the negative distance between the end-effector and the target goal. Hence, the state s_t is a set of distances between the agent and the target goals, such that $s_t \in \mathbb{R}^4$. In the Reacher environment, an episode ends when 150 steps have elapsed or the top of the arm, controlled by the agent, is within 7 cm of the goal.

4.1.3. Doom

Doom is a 3D navigation task, shown in Fig. 1 (right), where the agent must navigate among four rooms, trying to collect an item from one of the rooms, after which the episode ends.

Doors separate the rooms, which the agent must manually open. At each step, the agent receives a small negative reward of -0.01 , and upon finding the goal, it receives $+50$. The agent perceives the state and the four stacked RGB frames of shape (3, 84, 84) corresponding to color channels, width, and height. The agent has four actions available: forward, rotate left, rotate right, and activate door.

The episode ends when the agent reaches the item or exceeds 1250 steps. This setup and map are identical to that of Kulkarni et al. [39], except that a consistent action repeat was used across all available actions; this differs from the original implementation, where a different action repeat was used on a per-action basis, with some actions repeating once only and others several times, e.g., forward movements did not repeat, but rotation movements repeated five times.

4.2. Experiments

This section describes the experiments used to validate and test the DynSF model on Axes, Reacher, and Doom. Based on the original SF framework, all baseline models used in the experiments had an encoder, decoder, reward vector, and successor feature components. In contrast, the DynSF agents had an encoder, decoder, reward vector, and state-transition model.

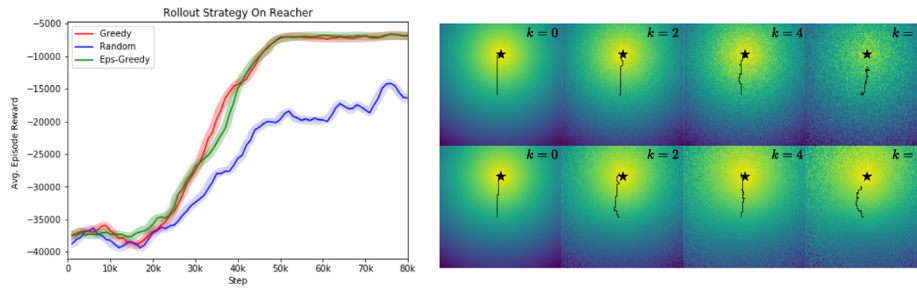


Fig. 2. **Left:** Learning curves on the Reacher environment with various rollout policies. **Right:** State-value maps of the Axes environment. The goal is shown as the black star, and the path taken by the agent from the origin as a black line. *Top row:* Oracle state-transition model and reward vector. *Bottom row:* Learned state-transition model and reward vector.

4.2.1. Rollout policy

Through the rollout policy experiment, the model's flexibility in using different policies for state visitation was examined. An evaluation of the different rollout policies used during the creation of $\psi(s, a, \bar{\pi}, \gamma)$ was performed, as defined in (5). Specifically, greedy, random, and ϵ -greedy policies were evaluated in the Reacher environment.

This simple experiment supports a more comprehensive use of flexible rollout policies for tasks such as safe RL and others on model-based environments.

4.2.2. Rollout length

As DynSF's policy was created dynamically with a variable rollout, the impact of the rollout length on the dynamic $\psi(s, a, \bar{\pi}, \gamma)$ was investigated. These experiments evaluated rollout lengths of $\{0, 2, 4, 8\}$ in the Axes environment. The aim was to understand how the hyperparameters affect the state-value function and whether a shorter rollout length can be used.

This analysis was performed using two variants of DynSF. The first variant used a fixed a priori state-transition model $\phi(s, a, \bar{\pi}, \gamma)$, which had already captured the optimal policy, called here Oracle DynSF. Therefore, Oracle DynSF had the perfect state transition model and had only to learn the reward vector. The other variant was that DynSF learned end-to-end, where either the state-transition model or the reward vector was learned. A state-value map for the Axes environment was produced, along with the path the agent took to reach the goal position.

4.2.3. Transfer with significant task change

A significant task change was used to further assess whether the flexibility offered by DynSF can better tackle transfer between different domains. The original SF framework algorithm and DynSF were both evaluated on the same grid world of Axes. The same reasoning was followed as in Lehnert et al. [40], causing the reward function to change drastically by changing the task, which meant that the goal was relocated to an opposite corner. The agent's start location was considered fixed for both tasks. The episode length and transfer capability of the agents were compared. Sufficient exploration was ensured by using a different ϵ -annealing approach than the original SF algorithm to make the comparison fair and to give the agents a chance to explore.

4.2.4. Performance evaluation

In the Reacher and Axes environment, DynSF was compared with the Q-learning algorithm and random baselines. In the Doom environment, the proposed DynSF was used to confirm that the method can handle an increasingly complex environment.

5. Results and discussion

This section presents the results of the experiments detailed in the previous section.

5.1. Rollout policy

Fig. 2 (left) shows the results of the various local policies evaluated on the Reacher environment: *greedy*, *random*, and ϵ -*greedy*. This experiment shows that the greedy and ϵ -greedy policies gave the best performance (close to 100%) in this environment. Although the random policy performed worse than the greedy and ϵ -greedy policies, it still delivered reasonable performance on the tasks. The completely random policy can be seen as forcing additional exploration because the resulting $\psi(s, a, \bar{\pi}, \gamma)$ function does not attempt to maximize reward during action selection. Each model's mean performance is reported on all plots as the average over runs with varied seeds, which also includes the standard deviation over all runs as a shaded area.

From the bands around each curve and general smoothness of the greedy and ϵ -greedy methods, we see that the optimal policy is quickly reached and maintained. In comparison, the random policy has greater variance towards the end of training.

Because the agent was trained with a decaying exploration rate, where the action from (5) was randomly selected with probability ϵ , the equal performance of the greedy and ϵ -greedy policies could have been expected. Both the greedy and ϵ -greedy policies explored at roughly the same rate under this scheduling. However, using an ϵ -greedy policy requires less computing because only the next latent state resulting from non-random actions needs to be predicted instead of all future proceeding states. This contrasts with the greedy policy, which expands each future state across the actions using the state-transition model. Therefore, the ϵ -greedy policy was used for the other experiments unless stated otherwise.

This experiment shows that both the greedy and ϵ -greedy policies were most performant during rollouts while training the model. However, a policy can be tailored to accommodate constraints, such as safe and guided explorations [31,41], since, in most applications, some properties or knowledge of the tasks can be exploited to speed up training and improve performance. Such applications could take advantage of the DynSF framework.

5.2. Rollout length

Fig. 2 (right) shows the state-value maps produced by varying the rollout length k in the Axes environment for $k = \{0, 2, 4, 8\}$. The map was created by taking the maximum value produced by (5) for the k steps under a local ϵ -greedy policy over all states in the Axes environment.

The reward function produced a clean gradient on a state-value map flowing between the agent and the goal. Note that the agent could solve the given tasks with up to $k = 4$ steps. After this, noise began to dominate the state-value estimates, and the agent could no longer solve the task. Interestingly, the learned state-transition model appears to have produced better

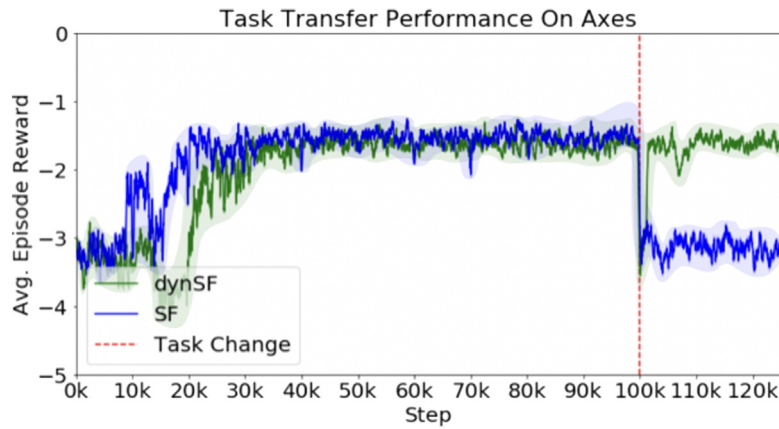


Fig. 3. DynSF and SF framework compared in a drastic task change. DynSF was able to transfer, but the original SF framework struggled to solve the new task. It is hypothesized here that this performance is the result of a more generalized state representation learned by the DynSF model, which has not specialized to a specific task.

quality estimates in the central area of the map, as evidenced by the smoother gradient. The oracle state-transition model did not have this smoothness around the most visited areas. Because the reward vector was identical, the authors believe that the latent state representation produced by the state-transition model is more resilient to the added noise of further rollout steps.

It can be concluded from these experiments that using many rollout steps might not be required and could decrease agent performance by introducing too much additional noise. Furthermore, there is a computational trade-off to be made because each added step forward requires additional prediction by the state-transition model. Therefore, the optimal number of rollout steps is the smallest number that does not degrade performance.

Many works report on the time and sample complexity of RL algorithms, such as [42,43]. The update equation for the standard value iteration iterates over all actions and overall possible next states. In the same vein, for DynSF, the value iteration sweeps through the number of rollouts and overall possible next states. Therefore, the intuition regarding this matter is that the time complexity of the DynSF algorithm is bounded by the number of rollout steps chosen. This is a clear trade-off between flexibility and computing. The standard SF model learns an amortized version of the successor features, implicitly trading flexibility for run-time performance. There are some potential savings in performance during the rollouts and some potential burdens inserted by the existence of the state transition model. Therefore, the overall performance is a trade-off between the number of rollouts and the additional transition model. However, as shown by this work, we believe the additional flexibility of the DynSF algorithm is worth the additional computing.

5.3. Transfer with significant task change

Fig. 3 shows the performance when transferring DynSF and the original SF framework within a substantial task change in Axes. The target goal changed drastically in location, from North(0,1) to South(0,-1). The agent's initial position was unchanged.

The DynSF framework achieved better results, which can be explained by analyzing the mathematical formulation that DynSF contributes. The SF and DynSF frameworks were derived by expanding the mathematical definition of Q-Learning. The next step is to acknowledge that a Q-Learning agent will nearly always have fewer parameters than an SF-based agent on the same task, implying that the Q-Learning agent must be learning some compressed form of the task structure. Most likely,

the Q-Learning model is learning a representation that is tuned to the particular task's reward structure, discount factor, and state dynamics, leading to a compressed representation where important information related to task change is tracked. This is a clear trade-off because the tracked information will be useful for related tasks. This is hypothesized here to be what leads the SF and DynSF models to generalize much better given a task change: the learned representations from the expanded formulation of such frameworks are much more general. The DynSF model takes this a step further by disentangling the future discounted reward, the discount factor, the acting policy, and the state dynamics to the task-related weights through the supervised learning state representation model, implying much greater generalization capabilities.

5.4. Performance evaluation

5.4.1. Axes

Fig. 4 (left) shows the results of DynSF against the original formulation of the SF framework and an oracle. The oracle is the DynSF algorithm but with a perfect environment model, hence the oracle label. Therefore, the rollouts will have no additive error. This shows the possible performance if DynSF learns a perfect state-transition model. The results were that all models solved the environment, but the dynamic variant did so faster and with less variance than the SF framework. In this case, only $k=3$ rollouts are considered in this experiment.

5.4.2. Reacher

Fig. 4 (right) shows the results of the proposed model against a DQN and a random baseline. Both the proposed model and the DQN could solve the environment. Although both models converged to the final solution, the proposed DynSF did so and remained very stable after convergence. We hypothesize that the DynSF model has greater stability because the learned features generalize better across the different states the model encounters as learning progresses. This is due to the loss function used to learn the hidden representation ϕ , which in the case of DynSF, focuses purely on reconstruction such that all information is present in the embedding, regardless of the induced state. In contrast, we believe the SF model discards information in the state embedding as it specializes in the specific task and states encountered by the policy.

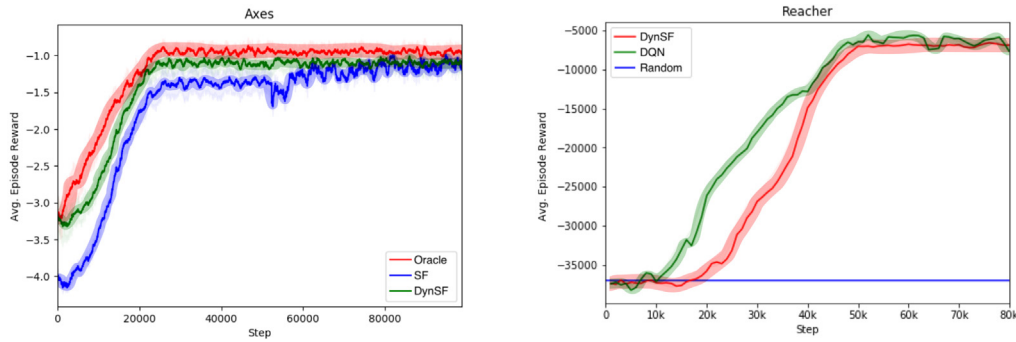


Fig. 4. Left: Learning curves on the Axes environment of an oracle model, the original SF formulation, and the proposed dynamic variant DynSF. Right: Learning curves in the Reacher environment of the proposed dynamic variant DynSF compared against DQN and random baselines.

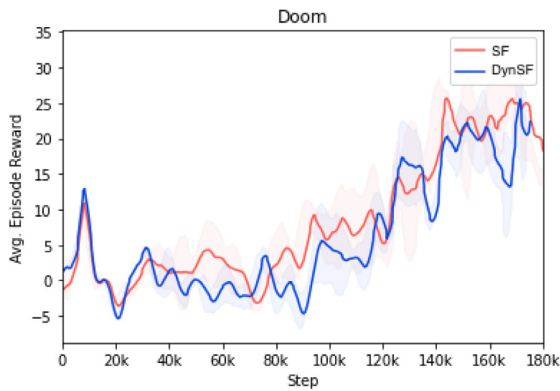


Fig. 5. Learning curves on the Doom environment of the proposed dynamic variant DynSF compared against the original SF formulation.

5.4.3. Doom

Fig. 5 shows the performance of the proposed model against the original formulation of the SF framework in the Doom environment. Fig. 5 illustrates that the proposed model can successfully navigate the environment to obtain the reward and is competitive with the original formulation of the SFs. Furthermore, it shows that the proposed method can scale up to complex environments and can do so from raw pixels.

6. Conclusion

This study has derived and presented a dynamic rollout representation of the SF framework called DynSF, which uses a state-transition model. The DynSF framework enables the setup of dynamic policies and discount factors, which brings flexibility to the original SF framework.

Mathematically, the new formulation brings a clearer insight into what is learned by the SF and DynSF frameworks. Experimentally, it has been shown that different rollout policies yield different performances. This supports a further investigation on DynSF to be used for guided exploration and Safe RL. Furthermore, the effects of rollout length were analyzed. Short rollouts are possible with reasonable performance, but lengthy rollouts can introduce noise to the training. With comparisons to baselines, it has been demonstrated that through the DynSF framework, RL can be performed with two supervised learning tasks, one for the state and another to reward prediction. Finally, given a drastic change in tasks, the DynSF framework adapts better to transfer than the original SF framework.

This result has significant implications, given that the learned state-model representation with dynamic policies and discount

factor sets up an on-the-fly flexible successor feature model and enables better task transfer by enabling different policies. The flexibility of the DynSF model, with both its mathematical specification and architectural simplicity, leaves room for natural extensions in future work.

Following from this, it can be hypothesized that the generalization, and therefore the quality, of the induced policy, could be greatly improved by looking to larger state-transition models such as Dreamer [44] or other generative models [45–47]. Such a model, given its large set of parameters, could possibly be generalized across many tasks in complex environments. Atari is an example of such an environment where the dynamics of certain tasks, such as Space Invaders and Demon Attack, are nearly identical. However, a more comprehensive analysis of the DynSF computational complexity must be assessed in order to support the feasibility of such an application.

Another line of future work, in a nearly opposite direction to that described in the previous paragraph, is the exploitation of state-transition model prediction noise to enhance the exploration ability of the agent. Indeed, the rollout length experiment described in Section 5.2 revealed that the compounding error causes the agent to explore *along* its path through the environment. The hypothesis is that this noise could be further enhanced by using it as an exploration bonus during learning, much as how other studies have used state-prediction or reconstruction as an exploration bonus for the agent [48,49].

Methods

General experimental setup

In this section, we discuss the model architecture used in each experiment.

This work follows the same methodology outlined in Deep SFs by [19] and Deep Q-Learning [3]. More exactly, the model was trained using batches sampled from a replay memory every four steps. All experiments involved learning on 64 samples in a batch, optimizing the model parameters with the Adam optimizer [50], using a learning rate of $5e-4$, a discount factor of 0.9, anneal ϵ over 50k steps and using a replay memory with 1e6 samples.

The Q-Learning agents followed an ϵ -greedy strategy where ϵ was decayed from 1.0 to some final value over the course of training. To stabilize learning, a model with frozen parameters is used to form the target in the Q-Learning updates. These models are updated to the current models parameters at fixed intervals throughout training.

Specifically, for the DynSF framework, except for the rollout length experiments, $k = 3$ rollouts were used. Moreover, except in the rollout policy experiments, where different policies were tested, an ϵ -greedy rollout policy was used the same ϵ as the agent. DynSF does not require a frozen model during learning.

Axes

In the Axes environment, the encoder and decoder we use is simply the identity function, $\phi_t = E(s_t) = s_t$ and $s_t = D(\phi_t) = \phi_t$, as the raw state is optimally coded.

In the baseline model, the SF branch is composed of 3 fully connected layers of 64, 64, and 8 units. The reward vector must match the encoded state, which in this case is 8. We use a hyperbolic tan activation function after all layers in the SF branch except at the final layer. The state-transition model we use is a ResNet as proposed in TreeQN [32,34] with a hidden dimension of 64 units.

Reacher

The Reacher environment also has the raw state optimally encoded such that we do not require an encoder or decoder in this environment. In the baseline model, we have a network with three fully connected layers of 64, 64, and 32 units with rectified linear activations after each layer, except the last.

Doom

As this environment is learned from raw pixels, we use a convolutional encoder and decoder. The encoder, decoder, reward vector, and SF used for the baseline model are identical to that of Kulkarni et al. [19]. Our model used an encoder, decoder, reward vector, and state-transition model. The encoder is identical to that used in DQN [3] with the decoder matched such that it can reconstruct an image of the correct dimensions. The state-transition model we use in Doom is the same as TreeQN [32,34] with a hidden dimension of 512 units.

Significant task change

This environmental setup is identical to that of the original Axes environment, but after 100k steps, the task changes. In this case, the original task the agent is trained on for the first 100k steps is directly above the agent in the north compass location. After 100k, the task changes where the goal is now below the agent, in the south compass location. We noticed the baseline struggled with this task change, and we increased the number of hidden units on the final layer such that the SF branch was composed of 3 fully connected layers of 64, 64, and 8 units. The DynSF model uses the same configuration as in the Axes task.

CRedit authorship contribution statement

Norman Tasfi: Conceptualization, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Eder Santana:** Conceptualization, Methodology, Writing – original draft. **Luisa Liboni:** Methodology, Formal analysis, Validation, Writing – original draft, Writing – review & editing. **Miriam Capretz:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This research was funded by The Natural Sciences and Engineering Research Council of Canada (NSERC): RGPIN-2021-04161 and by the Ontario Research Fund, Canada grant ORF-RE10-045.

References

- [1] David Silver, Aja Huang, Christopher J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, Demis Hassabis, Mastering the game of Go with deep neural networks and tree search, *Nature* 529 (2016) 484–503.
- [2] Matej Moravčík, Martin Schmid, Neil Burch, Viliam Lisý, Dustin Morrill, Nolan Bard, Trevor Davis, Kevin Waugh, Michael Johanson, Michael Bowling, DeepStack: Expert-level artificial intelligence in heads-up no-limit poker, *Science* (ISSN: 0036-8075) 356 (6337) (2017) 508–513, <http://dx.doi.org/10.1126/science.aam6960>.
- [3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, Demis Hassabis, Human-level control through deep reinforcement learning, *Nature* (ISSN: 00280836) 518 (7540) (2015) 529–533.
- [4] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, Koray Kavukcuoglu, Asynchronous methods for deep reinforcement learning, in: *International Conference on Machine Learning*, 2016, pp. 1928–1937.
- [5] Sergey Levine, Chelsea Finn, Trevor Darrell, Pieter Abbeel, End-to-end training of deep visuomotor policies, *J. Mach. Learn. Res.* 17 (1) (2016) 1334–1373.
- [6] Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, et al., Solving rubik's cube with a robot hand, 2019, arXiv preprint [arXiv:1910.07113](https://arxiv.org/abs/1910.07113).
- [7] Siddhartha Verma, Guido Novati, Petros Koumoutsakos, Efficient collective swimming by harnessing vortices through deep reinforcement learning, *Proc. Natl. Acad. Sci.* 115 (23) (2018) 5849–5854.
- [8] Paul Garnier, Jonathan Viquerat, Jean Rabault, Aurélien Larcher, Alexander Kuhnle, Elie Hachem, A review on deep reinforcement learning for fluid mechanics, *Comput. & Fluids* 225 (2021) 104973.
- [9] Qi Li, Xiang Meng, Fei Gao, Guorui Zhang, Weirong Chen, Kaushik Rajashekar, Reinforcement learning energy management for fuel cell hybrid system: A review, *IEEE Ind. Electron. Mag.* (2022).
- [10] Xiaosong Hu, Teng Liu, Xuwei Qi, Matthew Barth, Reinforcement learning for hybrid and plug-in hybrid electric vehicle energy management: Recent advances and prospects, *IEEE Ind. Electron. Mag.* 13 (3) (2019) 16–25.
- [11] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Sungmin Bae, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Anand Babu, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, Jeff Dean, Chip placement with deep reinforcement learning, 2020, [arXiv:2004.10746](https://arxiv.org/abs/2004.10746).
- [12] Richard S. Sutton, Andrew G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.
- [13] Yuanpeng Zhang, Kaijian Xia, Yizhang Jiang, Pengjiang Qian, Weiwei Cai, Chengyu Qiu, Lai Khin Wee, Dongrui Wu, Multi-modality fusion & inductive knowledge transfer underlying non-sparse multi-kernel learning and distribution adaption, *IEEE/ACM Trans. Comput. Biol. Bioinform.* (2022).
- [14] Honggui Han, Hongxu Liu, Cuili Yang, Junfei Qiao, Transfer learning algorithm with knowledge division level, *IEEE Trans. Neural Netw. Learn. Syst.* (2022).
- [15] Shohei Chiba, Hisayuki Sasaoka, Basic study for transfer learning for autonomous driving in car race of model car, in: *2021 6th International Conference on Business and Industrial Research, ICBIR, IEEE, 2021*, pp. 138–141.
- [16] Akshita Mittal, Purna Sowmya Munukutla, Visual transfer between atari games using competitive reinforcement learning, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2019.
- [17] André Barreto, Will Dabney, Rémi Munos, Jonathan J. Hunt, Tom Schaul, Hado P. van Hasselt, David Silver, Successor features for transfer in reinforcement learning, in: *Advances in Neural Information Processing Systems*, 2017, pp. 4055–4065.
- [18] Peter Dayan, Improving generalization for temporal difference learning: The successor representation, *Neural Comput.* (ISSN: 0899-7667) 5 (4) (1993) 613–624, <http://dx.doi.org/10.1162/neco.1993.5.4.613>.

- [19] Tejas D. Kulkarni, A. Saeedi, Simanta Gautam, S. Gershman, Deep successor reinforcement learning, 2016, arXiv [arXiv:1606.02396](#).
- [20] Chen Ma, Dylan R. Ashley, Junfeng Wen, Yoshua Bengio, Universal successor features for transfer reinforcement learning, 2020, arXiv preprint [arXiv:2001.04025](#).
- [21] Peter Dayan, Improving generalization for temporal difference learning: The successor representation, *Neural Comput.* 5 (4) (1993) 613–624.
- [22] Marlos C. Machado, Clemens Rosenbaum, Xiaoxiao Guo, Miao Liu, Gerald Tesauro, Murray Campbell, Eigenoption discovery through the deep successor representation, 2017, arXiv preprint [arXiv:1710.11089](#).
- [23] Jingwei Zhang, Jost Tobias Springenberg, Joschka Boedecker, Wolfram Burgard, Deep reinforcement learning with successor features for navigation across similar environments, in: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, IEEE, 2017, pp. 2371–2378.
- [24] Lucas Lehnert, Michael L. Littman, Successor features combine elements of model-free and model-based reinforcement learning, *J. Mach. Learn. Res.* 21 (2020) 1–53.
- [25] Andre Barreto, Diana Borsa, John Quan, Tom Schaul, David Silver, Matteo Hessel, Daniel Mankowitz, Augustin Zidek, Remi Munos, Transfer in deep reinforcement learning using successor features and generalised policy improvement, in: International Conference on Machine Learning, PMLR, 2018, pp. 501–510.
- [26] Majid Abdolshah, Hung Le, Thommen Karimpanal George, Sunil Gupta, Santu Rana, Svetha Venkatesh, A new representation of successor features for transfer across dissimilar environments, in: Marina Meila, Tong Zhang (Eds.), Proceedings of the 38th International Conference on Machine Learning, in: Proceedings of Machine Learning Research, Vol. 139, PMLR, 2021, pp. 1–9, URL <https://proceedings.mlr.press/v139/abdolshah21a.html>.
- [27] Hao Liu, Pieter Abbeel, Aps: Active pretraining with successor features, in: International Conference on Machine Learning, PMLR, 2021, pp. 6736–6747.
- [28] Laszlo Szoke, Szilárd Aradi, Tamás Bécsi, Péter Gáspár, Skills to drive: Successor features for autonomous highway pilot, *IEEE Trans. Intell. Transp. Syst.* (2022).
- [29] Lucas Lehnert, Michael L. Littman, Successor features support model-based and model-free reinforcement learning, 2019, CoRR Abs/1901.11437.
- [30] Igor Kuznetsov, Guided exploration in reinforcement learning via Monte Carlo critic optimization, 2022, arXiv preprint [arXiv:2206.12674](#).
- [31] Jiameng Fan, Wenchao Li, Safety-guided deep reinforcement learning via online gaussian process estimation, 2019, arXiv preprint [arXiv:1903.02526](#).
- [32] Gregory Farquhar, Tim Rocktäschel, Maximilian Igl, Shimon Whiteson, Treeqn and atreec: Differentiable tree-structured models for deep reinforcement learning, 2017, arXiv preprint [arXiv:1710.11417](#).
- [33] Junhyuk Oh, Xiaoxiao Guo, Honglak Lee, Richard L. Lewis, Satinder Singh, Action-conditional video prediction using deep networks in atari games, in: Advances in Neural Information Processing Systems, 2015, pp. 2863–2871.
- [34] Norman Tasfi, Miriam Capretz, Dynamic planning networks, 2018, arXiv preprint [arXiv:1812.11240](#).
- [35] Seyyed Amir Hadi Minoofam, Azam Bastanfard, Mohammad Reza Keyvanpour, TRCLA: a transfer learning approach to reduce negative transfer for cellular learning automata, *IEEE Trans. Neural Netw. Learn. Syst.* (2021).
- [36] Martin L. Puterman, Markov Decision Processes: Discrete Stochastic Dynamic Programming, first ed., John Wiley & Sons, Inc., USA, ISBN: 0471619779, 1994.
- [37] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, Dan Mané, Concrete problems in AI safety, 2016, arXiv preprint [arXiv:1606.06565](#).
- [38] Emanuel Todorov, Tom Erez, Yuval Tassa, Mujoco: A physics engine for model-based control, in: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE, 2012, pp. 5026–5033.
- [39] Tejas D. Kulkarni, Ardavan Saeedi, Simanta Gautam, Samuel J. Gershman, Deep successor reinforcement learning, 2016, arXiv preprint [arXiv:1606.02396](#).
- [40] Lucas Lehnert, Stefanie Tellex, Michael L. Littman, Advantages and limitations of using successor features for transfer in reinforcement learning, 2017, arXiv preprint [arXiv:1708.00102](#).
- [41] Sahisnu Mazumder, Bing Liu, Shuai Wang, Yingxuan Zhu, Xiaotian Yin, Lifeng Liu, Jian Li, Knowledge-guided exploration in deep reinforcement learning, 2022, arXiv preprint [arXiv:2210.15670](#).
- [42] Tor Lattimore, Marcus Hutter, Peter Sunehag, The sample-complexity of general reinforcement learning, in: International Conference on Machine Learning, PMLR, 2013, pp. 28–36.
- [43] Chi Jin, Zeyuan Allen-Zhu, Sebastien Bubeck, Michael I. Jordan, Is Q-learning provably efficient? *Adv. Neural Inf. Process. Syst.* 31 (2018).
- [44] Danijar Hafner, Timothy P. Lillicrap, Mohammad Norouzi, Jimmy Ba, Mastering atari with discrete world models, 2020, CoRR [arXiv:2010.02193](#), URL <https://arxiv.org/abs/2010.02193>.
- [45] S.M. Ali Eslami, Nicolas Heess, Theophane Weber, Yuval Tassa, Koray Kavukcuoglu, Geoffrey E. Hinton, Attend, infer, repeat: Fast scene understanding with generative models, 2016, CoRR [arXiv:1603.08575](#), URL <http://arxiv.org/abs/1603.08575>.
- [46] Tongzhou Wang, Phillip Isola, Understanding contrastive representation learning through alignment and uniformity on the hypersphere, 2020, CoRR [arXiv:2005.10242](#), URL <https://arxiv.org/abs/2005.10242>.
- [47] Joshua Robinson, Ching-Yao Chuang, Suvrit Sra, Stefanie Jegelka, Contrastive learning with hard negative samples, 2020, CoRR [arXiv:2010.04592](#), URL <https://arxiv.org/abs/2010.04592>.
- [48] P.-Y. Oudeyer, Jacqueline Gottlieb, Manuel Lopes, Intrinsic motivation, curiosity, and learning: Theory and applications in educational technologies, *Prog. Brain Res.* 229 (2016) 257–284.
- [49] Riley Simmons-Edler, Ben Eisner, Daniel Yang, Anthony Bisulco, Eric Mitchell, Sebastian Seung, Daniel Lee, Qxplorer: Q-learning exploration by maximizing temporal difference error, 2019.
- [50] Diederik P. Kingma, Jimmy Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint [arXiv:1412.6980](#).